In [1]:
```
import pandas as pd
from sklearn import preprocessing, model_selection

file_url = ('https://raw.githubusercontent.com/'
            'PacktWorkshops/'
            'The-Applied-Artificial-Intelligence-Workshop/'
            'master/Datasets/car.csv')
df = pd.read_csv(file_url)

print("Data loaded, shape:", df.shape)
print(df.head())
```

```
Data loaded, shape: (1728, 7)
  buying maintenance doors persons luggage_boot safety  class
0  vhigh       vhigh     2       2        small    low  unacc
1  vhigh       vhigh     2       2        small    med  unacc
2  vhigh       vhigh     2       2        small   high  unacc
3  vhigh       vhigh     2       2          med    low  unacc
4  vhigh       vhigh     2       2          med    med  unacc
```

In [2]:
```
def encode(data_frame, column):
    label_encoder = preprocessing.LabelEncoder()
    label_encoder.fit(data_frame[column].unique())
    return label_encoder.transform(data_frame[column])

for column in df.columns:
    df[column] = encode(df, column)

print("All columns encoded:")
print(df.head())
```

```
All columns encoded:
   buying  maintenance  doors  persons  luggage_boot  safety  class
0       3            3      0        0             2       1      2
1       3            3      0        0             2       2      2
2       3            3      0        0             2       0      2
3       3            3      0        0             1       1      2
4       3            3      0        0             1       2      2
```

In [3]:
```
label = df.pop('class')

print("Label shape:", label.shape)
print("Features shape:", df.shape)
```

```
Label shape: (1728,)
Features shape: (1728, 6)
```

In [4]:
```
features_train, features_test, label_train, label_test = \
    model_selection.train_test_split(df, label, test_size=0.1, random_state=88)

print("Train:", features_train.shape, "Test:", features_test.shape)
```

```
Train: (1555, 6) Test: (173, 6)
```

In [5]:
```
from sklearn.tree import DecisionTreeClassifier
```

```
decision_tree = DecisionTreeClassifier(random_state=88)
decision_tree.fit(features_train, label_train)
dt_predictions = decision_tree.predict(features_test)

print("Decision tree trained. Depth:", decision_tree.get_depth())
print("First 10 predictions:", dt_predictions[:10].tolist())
```

```
Decision tree trained. Depth: 14
First 10 predictions: [1, 1, 2, 1, 0, 2, 3, 2, 2, 2]
```

In [6]:
```
print("Decision tree accuracy:", decision_tree.score(features_test, label_test))
```

```
Decision tree accuracy: 0.953757225433526
```

In [7]:
```
from sklearn.metrics import classification_report

print(classification_report(label_test, dt_predictions, zero_division=0))
```

```
              precision    recall  f1-score   support

           0       0.87      0.98      0.92        42
           1       0.89      0.89      0.89         9
           2       0.99      0.96      0.98       114
           3       1.00      0.75      0.86         8

    accuracy                           0.95       173
   macro avg       0.94      0.89      0.91       173
weighted avg       0.96      0.95      0.95       173
```

In [8]:
```
print("Reusing the split from Activity 4.01")
print("Train samples:", len(features_train), "| Test samples:", len(features_test))
print("Features:", list(features_train.columns))
```

```
Reusing the split from Activity 4.01
Train samples: 1555 | Test samples: 173
Features: ['buying', 'maintenance', 'doors', 'persons', 'luggage_boot', 'safety']
```

In [9]:
```
from sklearn.ensemble import RandomForestClassifier

random_forest_classifier = RandomForestClassifier(n_estimators=100,
                                                  max_depth=6,
                                                  random_state=168)
print("Random forest created:")
print(random_forest_classifier)
```

```
Random forest created:
RandomForestClassifier(max_depth=6, random_state=168)
```

In [10]:
```
random_forest_classifier.fit(features_train, label_train)

print("Random forest trained. Trees:", len(random_forest_classifier.estimators_))
print("Train accuracy:", random_forest_classifier.score(features_train, label_train
print("Test accuracy: ", random_forest_classifier.score(features_test, label_test))
```

```
Random forest trained. Trees: 100
Train accuracy: 0.9118971061093247
Test accuracy:  0.8439306358381503
```

In [12]:
```
from sklearn.metrics import confusion_matrix

rf_preds_test = random_forest_classifier.predict(features_test)
print("First 10 predictions:", rf_preds_test[:10].tolist())

print("Confusion matrix (rows = true, columns = predicted):")
print(confusion_matrix(label_test, rf_preds_test))
```

```
First 10 predictions: [0, 0, 2, 0, 0, 2, 3, 2, 2, 2]
Confusion matrix (rows = true, columns = predicted):
[[ 32   0  10   0]
 [  8   0   0   1]
 [  5   0 109   0]
 [  3   0   0   5]]
```

In [13]:
```
print(classification_report(label_test, rf_preds_test, zero_division=0))
```

```
              precision    recall  f1-score   support

           0       0.67      0.76      0.71        42
           1       0.00      0.00      0.00         9
           2       0.92      0.96      0.94       114
           3       0.83      0.62      0.71         8

    accuracy                           0.84       173
   macro avg       0.60      0.59      0.59       173
weighted avg       0.80      0.84      0.82       173
```

In [14]:
```
rf_varimp = random_forest_classifier.feature_importances_
print("Random forest feature importances:")
print(rf_varimp)
```

```
Random forest feature importances:
[0.12676384 0.10366314 0.02119621 0.35266673 0.05915769 0.33655239]
```

In [15]:
```
from sklearn.ensemble import ExtraTreesClassifier

extra_trees_classifier = ExtraTreesClassifier(n_estimators=100,
                                               max_depth=6,
                                               random_state=168)
extra_trees_classifier.fit(features_train, label_train)

print("Extra trees trained:")
print(extra_trees_classifier)
print("Train accuracy:", extra_trees_classifier.score(features_train, label_train))
print("Test accuracy: ", extra_trees_classifier.score(features_test, label_test))
```

```
Extra trees trained:
ExtraTreesClassifier(max_depth=6, random_state=168)
Train accuracy: 0.8836012861736334
Test accuracy:  0.815028901734104
```

In [16]:
```
et_preds_test = extra_trees_classifier.predict(features_test)
print("First 10 predictions:", et_preds_test[:10].tolist())
```

```
print("Confusion matrix (rows = true, columns = predicted):")
print(confusion_matrix(label_test, et_preds_test))
```

```
First 10 predictions: [0, 0, 2, 0, 2, 2, 3, 2, 2, 2]
Confusion matrix (rows = true, columns = predicted):
[[ 28   0  14   0]
 [  9   0   0   0]
 [  2   0 112   0]
 [  7   0   0   1]]
```

In [17]:
```
print(classification_report(label_test, et_preds_test, zero_division=0))
```

```
              precision    recall  f1-score   support

           0       0.61      0.67      0.64        42
           1       0.00      0.00      0.00         9
           2       0.89      0.98      0.93       114
           3       1.00      0.12      0.22         8

    accuracy                           0.82       173
   macro avg       0.62      0.44      0.45       173
weighted avg       0.78      0.82      0.78       173
```

In [18]:
```
et_varimp = extra_trees_classifier.feature_importances_
print("Extra trees feature importances:")
print(et_varimp)
```

```
Extra trees feature importances:
[0.08844544 0.0702334  0.01440408 0.37662014 0.05965896 0.39063797]
```

In [ ]:
```

```